

Ejercicio 3 de la práctica 1

Punto a del ejercicio 3

n = a.length

void ordenarPorBurbujeo(**int** a[]) {

for (**int** i = 1; i < a.length; i++) { -> 1 + 4 n

for (**int** j = 0; j < a.length-1; j++) { -> 1 + 5 n

if (a[j] > a[j+1]) -> 4 operaciones

 swap(a[j], a[j+1]); -> 6 operaciones

 }

 }

}

Void swap(int n, int m){

 int aux = n;

 n = m;

 m = aux;

} **Son 3 instrucciones**

$1 + 4n + (n-1) * [1 + 5n + (n-1) 10]$

$10n^2 + n + 5n^2 + 1 + 4n = 15n^2 + 5n + 1 = f(n)$

Encontrar c y n_0 , de manera que se cumpla que:

$15n^2 + 5n + 1 < c n^2$, para todo $n > n_0$

Conjeturo $g(n) = n^2$ y tomo $c = 16$.

Elijo c=16

Ahora encuentro a partir de qué n_0 se cumple que:

$15n^2 + 5n + 1 < 16n^2$

$15n^2 + 5n + 1 - 15n^2 < 16n^2 - 15n^2$

$5n + 1 < n^2$

$5n + 1 < n * n$

Se cumple a partir $n_0=6$

$$5 * 6 + 1 = 31 < 36 = 6 * 6$$

$$5 * 5 + 1 = 26 > 25 = 5 * 5$$

Entonces, por definición de complejidad, $f(n)$ pertenece a $O(n^2)$ para todo $n > 5$

Existe $g(n) = n^2$ y $c = 16 /$ para todo $n > n_0 = 5$ se cumple que f es $O(n^2)$

Punto b del ejercicio 3

$n = a.length$

```
void ordenarPorBurbujeo(int a[]) {  
    for (int i = 1; i < a.length; i++) { -> 1 + 4n  
  
        for (int j = 0; j < a.length-i; j++) { -> ? (*) usa i para terminar sus ciclos  
  
            if (a[j] > a[j+1]) -> 4 operaciones  
                swap(a[j], a[j+1]); -> 6 operaciones  
  
        }  
    }  
}
```

Recordar que...

Si la suma es hasta $n-1$

$$f(n) = \sum_{i=1}^{n-1} i = (n * (n - 1) * 1/2)$$

Entonces, la $f(n)$ del punto (b) sería:

$$\underbrace{10 (n * (n-1) / 2)}_{(a)} + \underbrace{(n-1)}_{(b)} + \underbrace{5 (n * (n-1) / 2 + (n-1))}_{(c)} + \underbrace{1 + 4n}_{(d)} =$$

a) operaciones dentro del for interno

b) la asignación del for interno (int j=0)

c) operaciones en la definición del for interno (sin contar int j=0)

El $(n-1) \rightarrow$ viene de ejecutar una vez más la comparación y el $i++$ del for (recuerden que lo hace para terminar las iteraciones)

d) operaciones en la definición del for externo (idem. Punto c, ya que en lugar de $n-1$ las realiza n veces)

$$\begin{aligned}
&= 15 (n * (n-1) / 2) + n - 1 + 5 (n - 1) + 1 + 4n \\
&= 15 (...) + (n - 1 + 1 + 4n + 5n - 5) = 15 (n^2 - n) * 1/2 + 10n - 5 \\
&= 7,5 n^2 + (10n - 7,5n) - 5 = \underline{7,5 n^2 + 2,5 n - 5} \rightarrow 8 n^2 + 3 n - 5 \\
&= 8 n^2 + 3 n - 5
\end{aligned}$$

El algoritmo sigue siendo de orden cuadrático. Se puede notar que la constante que multiplica al término en n^2 , es casi la mitad de la obtenida en el punto a.

$$f(n) = 8 n^2 + 3 n - 5$$

Demostrar por algebra de ordenes

$$f(n) = 8 n^2 + 3 n - 5$$

$$O(f(n)) = O(8 n^2) + O(3 n) + O(-5) = // R2$$

$$O(8) \cdot O(n^2) + O(3) \cdot O(n) + O(-5) = // R3$$

$$O(1) \cdot O(n^2) + O(1) \cdot O(n) + O(1) = // R4$$

$$O(1 \cdot n^2) + O(1 \cdot n) + O(1) = // R3$$

$$O(n^2) + O(n) + O(1) =$$

$$= O(\max\{n^2, n, 1\}) = O(n^2) // R2$$

Justificación : n^2 tiene exponente 2, n tiene exponente 1 y 1 (que sería n^0) tiene exponente 0 $\rightarrow$ elijo el mayor exponente.

$\rightarrow f$ es **$O(n^2)$**

Dice R1:

$$O(n) < O(n^2) < \dots O(n^k) \text{ elijo el que tiene mayor } k, \text{ o sea } k=2$$

Sobre la R1

$$\text{Raíz}(n) \rightarrow n^{1/2} \text{ por lo tanto } k=1/2$$

$$f(n) = \sum_{i=1}^{n-1} i = (n * (n - 1) * 1/2)$$

Ejemplo: suma de Gauss para n = 10

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 =$$

$$(1 + 10) + (2 + 9) + (3 + 8) + (4 + 7) + (5 + 6) = 11 * 5 = 55$$

Fórmula general:

$$(n+1) * n / 2 = (n^2 + n) / 2 = \frac{1}{2} n^2 + \frac{1}{2} n \text{ pertenece a } O(n^2)$$